

Tree of Thoughts: Deliberate Problem Solving with Large Language Models

Re-implementation Report · Jane Tenecota-Villa, Jade Lee, Lauren Ah-Hot
CS 4782 Deep Learning · Cornell University · Based on Yao et al., NeurIPS 2023
GitHub: github.com/yunseoo2/deep-learning-tree-of-thought

1. Introduction

Large language models (LLMs) generate text autoregressively, committing to each token in a strict left-to-right order with no ability to backtrack. While this works well on straightforward tasks, it struggles on problems that require planning, exploring alternative paths, or recovering from early mistakes.

Yao et al. introduced **Tree of Thoughts (ToT)** in their NeurIPS 2023 paper, a framework that treats LLM inference as a search over a tree of intermediate reasoning steps. Instead of committing to a single chain of reasoning, the model generates multiple candidate "thoughts," evaluates them as sure/maybe/impossible, and uses search strategies such as breadth-first search (BFS) or depth-first search (DFS) to navigate the tree, backtracking when needed. This adds a more deliberate planning layer on top of standard autoregressive generation.

The original ToT paper evaluates the framework on Game of 24, Creative Writing, and Mini Crosswords, achieving a 74% success rate on Game of 24 compared to standard Input-Output (IO) prompting at 7.3% and Chain-of-Thought (CoT) prompting at 4% using GPT-4. CoT prompting improves over IO on many tasks by asking the model to reason step by step before giving an answer, but it still commits to a single reasoning path without backtracking. Our project reproduces the Game of 24 experiments using GPT-4o, re-implements the BFS-based ToT framework with a few-shot evaluator, and extends the work by analyzing how few-shot versus zero-shot evaluators affect overall performance.

2. Chosen Result

We targeted the Game of 24 main result (Table 2 of the paper): the comparison between IO, CoT, and ToT prompting. This result directly supports the paper's central claim that search-based reasoning substantially outperforms standard prompting on tasks requiring exploration. The paper reports IO: 7.3%, CoT: 4%, ToT (b=5): 74%.

We also aimed to reproduce the error analysis insight from Figure 3(b), which shows that roughly 60% of CoT failures occur at the very first step, which suggests that strict left-to-right decoding struggles on tasks like this. Beyond replication, we designed an evaluator ablation (few-shot vs. zero-shot) as an independent experiment to understand the degree to which ToT's performance depends on the evaluator versus the search structure itself.

3. Methodology

Dataset and Model. We randomly sampled 50 puzzles from the 4nums.com Game of 24 dataset (hard games, indices 901-1,000), reducing from the paper's 100 due to budget constraints. We used GPT-4o via the OpenAI API. The paper used a May 2023 GPT-4 checkpoint; GPT-4o is the current successor and is both more capable and more cost-efficient.

ToT Implementation. We implemented BFS with depth $T=3$ (three intermediate arithmetic equations) and beam size $b=5$. At each tree node a *propose prompt* asks the model to enumerate candidate next steps given the remaining numbers. A separate *value prompt* classifies each candidate as sure/maybe/impossible with respect to reaching 24. We sample values 3 times per thought and

combine scores to retain the top-b candidates per level.

Baselines. IO prompting (5-shot, direct answer) and CoT prompting (5-shot with 3 intermediate equations) serve as baselines, each sampled 5 times per puzzle. A solution is correct if it is a valid arithmetic equation using each input number exactly once and evaluating to 24, verified automatically by evaluating the generated expression through a program.

Modifications from the paper. (1) 50 vs. 100 puzzles due to API cost. (2) GPT-4o instead of GPT-4. (3) We wrote our own 7-example few-shot evaluator rather than reusing the paper's prompts. (4) Evaluator ablation: over 20 puzzles, the proposer and beam were held fixed while comparing our few-shot evaluator against a zero-shot evaluator, measuring both accuracy and wall-clock time per problem.

4. Results & Analysis

Method	Success Rate	Cost / Problem	Time / Problem
IO (5-shot)	10.0%	~\$0.0024	~1.2s
CoT (5-shot)	24.0%	~\$0.0022	~1.3s
ToT BFS (b=5)	88.0%	~\$0.1020	~86.0s

Table 1: Re-implementation results on 50 Game of 24 puzzles (GPT-4o).

Main results. Our re-implementation confirms the paper's key finding: ToT (88%) >> CoT (24%) >> IO (10%). Absolute numbers are higher than the paper's (74% / 4% / 7.3%), which we attribute to GPT-4o being a stronger model than the original May 2023 GPT-4 checkpoint. However, the relative ordering and large performance gap between ToT and standard prompting methods are fully replicated.

Evaluator ablation. This ablation was initially motivated by an implementation mistake on our end. In our first version, we used a zero-shot evaluator because the paper's main text largely assumes the evaluator setup without emphasizing how important the few-shot examples are. It was only after reviewing the official implementation code that we realized the evaluator itself used few-shot prompting. This led us to directly compare few-shot and zero-shot evaluation on a 20-puzzle subset. Replacing our 7-example few-shot evaluator with a zero-shot evaluator reduced accuracy from 95% to 25%, suggesting that evaluator quality plays a major role in ToT's performance. The zero-shot evaluator frequently over-pruned "maybe" states, causing promising search paths to be discarded too early. Runtime decreased slightly (70s vs. 97s) since the search tree became smaller, but at a major cost to accuracy. While the few-shot evaluator increased cost by roughly \$0.08 per problem, it also improved accuracy by 70 percentage points, making it a far more cost-effective design choice.

Cost analysis. ToT costs roughly 45x more per problem than IO or CoT (~\$0.10 vs. ~\$0.002) due to its search-based structure. However, our implementation is substantially cheaper than the original paper's setup, which reported roughly \$0.74 per Game of 24 problem using the May 2023 GPT-4 API. Using GPT-4o, our full 50-puzzle run cost about \$5 total. Cost per correct solution is more comparable across methods, since IO and CoT spend many attempts on failed solutions.

5. Reflections

Replication holds. Tree of Thoughts (ToT) consistently outperforms both CoT and IO on Game of 24, preserving the same qualitative ordering reported by Shunyu Yao et al. On our 50-problem sample, we observe 88% (ToT) vs. 24% (CoT) vs. 10% (IO). One notable difference is that the

original paper reports CoT (4%) performing below IO (7.3%), which the authors attribute to CoT committing early to incorrect reasoning paths, while random direct guessing in IO occasionally succeeds by chance. We do not reproduce this inversion, likely because GPT-4o follows intermediate reasoning prompts more reliably than the original May 2023 GPT-4 checkpoint.

The few-shot evaluator matters. Removing few-shot demonstrations from the evaluator prompt caused performance to drop sharply from 95% to 25%. While the original paper does not explicitly isolate this component, our results suggest that evaluator quality is a major contributor to ToT's success, rather than just a small implementation detail. In particular, the evaluator's ability to correctly classify ambiguous "maybe" states depends heavily on concrete examples.

Cost is the main tradeoff. The largest downside of ToT is its search overhead. Exploring and evaluating multiple reasoning paths dramatically improves accuracy, but also increases runtime and API usage compared to standard prompting methods. Still, newer models like GPT-4o already make this tradeoff much more practical than it was at the time of the original paper, and future models will likely continue reducing both inference time and cost. This suggests that search-based reasoning methods like ToT may become increasingly practical as model efficiency improves.

Future work. On the search side, replacing BFS with DFS and early stopping on high-confidence paths could preserve much of the accuracy while reducing cost, since BFS expands all candidates at every level. On the prompting side, it remains unclear whether 7 few-shot evaluator examples are actually necessary; testing different example counts (e.g., 3, 5, 7) could identify a cheaper configuration that still preserves most of the accuracy gain. Both directions point toward the broader challenge of making ToT more practical while maintaining its reasoning performance.

6. References

- [1] Yao, S., Yu, D., Zhao, J., Shafran, I., Griffiths, T. L., Cao, Y., & Narasimhan, K. (2023). Tree of Thoughts: Deliberate Problem Solving with Large Language Models. *NeurIPS 2023*. arXiv:2305.10601.
- [2] OpenAI (2024). GPT-4o. <https://openai.com/index/hello-gpt-4o/>
- [3] 4nums.com Game of 24 dataset / ToT code. <https://github.com/princeton-nlp/tree-of-thought-llm>